Jnana and the Next Open Source Revolution

Constitutional Compilation, AI Containment, and Operating-System Sovereignty

Jnana proposes a constitutional transformation substrate for this problem.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Open source made source code inspectable, modifiable, redistributable, and forkable under declared licenses and social institutions. AI coding agents change a different variable: the volume, speed, and reach of proposed transformation. When generation, dependency choice, shell execution, build mutation, and deployment occur through tools with broad ambient access, source visibility alone does not reveal who authorized an effect, which constraints applied, or whether the transformation can be reconstructed and refused.

Jnana proposes a constitutional transformation substrate for this problem. AI is treated as a proposer rather than an authority. Proposed changes are represented in governed intermediate form, evaluated under versioned law, lowered through admitted transformation machinery, mediated at effect boundaries, and bound to receipts. Refusal, deferment, branch, rollback, and recovery are legitimate outcomes.

This paper presents Jnana as a possible extension of open-source sovereignty, not a replacement for Linux, Git, maintainers, review, testing, licensing, or community governance. It turns "open power" into specific requirements: inspectable authority, bounded capability, reconstructable transformation, portable governance, provider exit, visible exceptions, and negative power. It also states the assurance boundary. An AI cannot execute arbitrary system effects only if every relevant path is isolated or mediated; deterministic replay exists only under a declared environment closure; hardware sovereignty extends only to the verified hardware and firmware boundary.

The paper integrates Jnana, Jnana II, MCIR, Governed Agentic Coding, Governance Coverage Gaps, GMB, and the universal GSM into an incremental adoption program. No implementation, containment result, Linux integration, hardware proof, or publication authority is claimed.

Source, Evidence, and Standing

This is an AI-assisted integrated edition derived from one exact author manuscript. Its custody manifest is
`work/paper-saturation-v1/evidence/jnana-and-the-next-open-source-revolution/integration-custody-manifest.v0.1.json`,
SHA-256
`F672237776C2EE74897BCB3CEB4CA68F9918610129222E18B136B50C32BDDF49`.
The source remains unchanged.

The title, open-source thesis, Jnana vision, operating-system and hardware scenarios, AI-as-proposer role, proposed workflow, and invitation are author material. Definitions, formal boundaries, threat model, adoption ladder, evaluation, counterevidence, corpus routing, and decisions added here are AI editorial or synthetic candidates until adopted. No external research was conducted.

1. The Problem Is Transformation Authority

Code visibility does not by itself answer:

- who or what proposed a change;
- which source, dependency, model, tool, and policy versions contributed;
- who possessed authority to approve it;
- which capabilities were granted during construction and execution;
- which effects occurred;
- what was refused or omitted; and
- whether the result can be reconstructed without the original provider.

Open-source projects already use review, testing, signed commits, reproducible-build techniques, sandboxing, and community governance to address parts of this problem. The source's stronger claims should be narrowed: these practices are not universal, and no claim that conventional systems lack all substrate-level governance is established here.

Jnana targets the remaining structural problem: transformation authority is often external to the artifact being transformed and weakly bound to the resulting effect.

2. Bounded Definition of Jnana

Jnana is a proposed **constitutional transformation substrate** that coordinates:

- **MCIR:** a canonical candidate representation carrying semantic and governance commitments;
- **MCIR-Law:** versioned admissibility rules over proposals and transformation regimes;

- **constitutional invariants:** higher-order limits on ordinary law mutation;
- **PEIR:** a source-defined procedural execution representation whose current corpus identity and conformance remain to be adjudicated;
- **Domain Effects:** declared classes and membranes for contact with system resources;
- **ReceiptLog:** evidence and lineage for decisions, transformations, effects, and revision; and
- **recovery epochs:** explicit emergency, rollback, and post-exception review.

The biological language in the Jnana family is metaphorical architecture, not evidence that software is an organism. This paper emphasizes the transformation-substrate interpretation because it connects directly to open-source practice.

3. AI as Proposer Is an Enforced Role

"AI as proposer" is not achieved by naming the model a proposer. It requires an execution architecture in which model output lacks direct standing and cannot bypass the authorization path.

The minimum invariant is:

> Model output may describe or propose an effect, but only an independently authorized and mediated realization may produce that effect.

This requires separation among:

- model inference and effect execution;
- proposal identity and author/maintainer standing;
- capability request and capability grant;
- generated code and admitted artifact;
- review evidence and decision authority; and
- execution receipt and semantic truth.

If an agent can write files, run a shell, alter CI, invoke a provider, or modify credentials outside the governed route, the proposer boundary is incomplete. Governance Coverage Gaps supplies the diagnostic for those escape paths.

4. The Constitutional Transformation Path

A bounded transformation can be represented as:

$$T = (I, P, A, L, D, X, W, V),$$

where I identifies exact inputs and dependencies, P the proposal, A authority, L applicable law, D the decision, X realized effects, W receipts and witnesses, and V verification and revision state.

The intended path is:

1. bind exact source, tool, dependency, model, and environment identities;
2. represent the proposed transformation and its requested capabilities;
3. resolve authority and applicable law;
4. admit, refuse, defer, branch, or escalate;
5. lower through an admitted compiler or transformation regime;
6. mediate each external effect at its grant and invocation boundary;
7. record result, partial failure, rollback, and residual state;
8. verify claimed properties under a declared observation regime; and
9. revise without erasing predecessor custody.

The path does not make a proposal correct. It makes the conditions under which it can bind inspectable.

5. Negative Power and Refusal

Open-source sovereignty includes the ability to inspect, modify, fork, and exit. Jnana adds **negative power**: the ability of a user, maintainer, policy, or constitutional layer to prevent a proposed transformation from binding.

Refusal must be more than an error code. It should bind:

- the exact proposal and requested capability;
- the law and authority under which it was evaluated;
- the reason and evidence class;
- whether revision, appeal, alternate realization, or branch is available; and
- whether any effect occurred before refusal.

Negative power becomes hollow if maintainers cannot inspect the law, providers can change hidden dependencies, emergency paths bypass receipts, or generated artifacts cannot be reconstructed.

6. Operating-System Integration Is a Profile, Not the Kernel

Jnana need not replace Linux or become a monolithic kernel. It can be realized as profiles around selected transformation and effect boundaries:

- source and dependency intake;
- build and package execution;
- module or extension admission;
- service installation and activation;
- agent tool grants;
- filesystem, process, network, and device effects;
- configuration-to-execution transitions; and
- update, rollback, repair, and recovery.

Kernel configuration in MCIR, gated module insertion, declared syscall surfaces, and AI-proposed patches are candidate configurations. They remain hypothetical until a concrete boundary, threat model, integration design, and court of tests exist.

No constitutional layer can govern effects it cannot observe or mediate. Lower firmware, management engines, peripheral controllers, cloud services, and physical access remain outside the claim unless incorporated into the verified boundary.

7. Hardware and Provider Sovereignty

Hardware sovereignty is not achieved by hashing firmware alone. A trustworthy profile may require:

- reproducible or independently reconstructable artifacts;
- measured and verified boot where supported;
- explicit backend, ABI, instruction, device, and firmware assumptions;
- signed provenance whose key authority is governed;
- update and rollback policy;
- anti-rollback or explicit downgrade semantics;
- physical and recovery threat boundaries; and
- evidence that the realized hardware behavior corresponds to the declared artifact.

Provider sovereignty likewise requires exportable source, law, schemas, identity data, receipts, dependencies, and reconstruction instructions. A portable receipt that still depends on a proprietary unavailable model or service does not establish provider exit.

8. What Jnana Adds to Open Source

Jnana's strongest proposed contribution is not simply machine-checkable governance. It is a collaboration surface in which contributors can inspect and contest:

- transformation regimes and compiler passes;
- capability and effect classes;
- authority and delegation;
- identity and equivalence law;
- constitutional invariants and amendment rules;
- evidence and replay classifications;
- exceptions, break-glass events, and recovery; and
- the correspondence between source, artifact, and realized effect.

This can make authority more visible. It can also increase complexity, centralize schema control, exclude contributors, or encode a flawed constitution more durably. The architecture must preserve human contestability and forkability rather than replacing social governance with an opaque policy engine.

9. Incremental Adoption Ladder

The source presents an integrated vision. Adoption need not begin with a complete operating system.

- **Level:** 0
  Bounded adoption target: exact source/result custody and receipts
  Claim ceiling: traceability only
- **Level:** 1
  Bounded adoption target: capability/effect inventory and visible grants
  Claim ceiling: observed inventory, not complete mediation
- **Level:** 2
  Bounded adoption target: proposal/approval separation and refusal
  Claim ceiling: bounded authority separation
- **Level:** 3
  Bounded adoption target: admitted transformation regime with MCIR artifacts
  Claim ceiling: conformance only for tested transformations
- **Level:** 4
  Bounded adoption target: mediated build, package, service, and agent effects
  Claim ceiling: coverage limited to demonstrated paths
- **Level:** 5
  Bounded adoption target: OS and hardware integration profile
  Claim ceiling: assurance limited to verified platform boundary

Each level needs its own evidence. A project should not claim Level 5 because it has Level 0 receipts.

10. Corpus Integration

Jnana supplies the original governed-genome architecture. **Jnana II** reframes it as a constitutional transformation substrate. This paper projects those ideas into open-source, operating-system, and hardware collaboration. It does not replace either work.

MCIR supplies the machine-legible transformation carrier. **Governed Agentic Coding** applies related authority and evidence disciplines to agent-produced software. **Governance Coverage Gaps** tests whether any capability path escapes the proposed membrane. **Constitutional Computing Architecture** is an adjacent, identity-held architecture that situates the layers more broadly and is not merged here.

The **Governed Meaning Base (GMB)** can retain identities, law versions, capabilities, effects, evidence, conflicts, exceptions, and revision lineage. The universal **GSM** can type their relations while distinguishing proposal, authority, artifact, realization, evidence, and standing.

11. Realization Funnel

$$\begin{aligned} & P \\ & \to S \\ & \to CM_1 \\ & \to CM_2 \\ & \to IC \\ & \to IR \\ & \to E \\ & \to R \end{aligned} \quad .$$

- **Possibility:** enumerate integration profiles, effect boundaries, governance models, contributor roles, and provider choices.
- **Specification:** define authority, law, capability, effects, receipts, refusal, recovery, and claim ceilings.
- **CM1:** retain admissible configurations, including minimal, centralized, costly, or undesirable ones.
- **CM2:** prefer configurations by sovereignty, containment, inspectability, accessibility, reconstruction, performance, and community governance.
- **IC:** select an **Instance Constitution** or **Instance Configuration** for one project or platform. The expansions are interchangeable.
- **IR:** produce an **Instance Realization** or **Implementation Realization** such as schemas, compilers, gates, membranes, stores, adapters, and user interfaces. The expansions are interchangeable.
- **Evidence:** run custody, bypass, capability, replay, build, effect, rollback, provider-exit, and human-governance tests.
- **Revision:** amend law and architecture through explicit authority, successor identity, migration, and receipts.

This paper reaches Specification and proposes CM1/CM2 and adoption criteria. It does not select a complete IC or produce an IR.

12. Evaluation Program

The proposal should be evaluated through: (1) untrusted AI proposals; (2) dependency insertion; (3) file, process, network, credential, and device effects; (4) build and installation hooks; (5) denied and deferred capability; (6) direct-path and emergency bypass attempts; (7) partial failure and rollback; (8) deterministic and equivalence-class replay; (9) compiler and policy upgrades; (10) provider removal and reconstruction; (11) firmware and boot boundaries; (12) contributor comprehension; (13) maintainership and fork governance; (14) accessibility and latency; and (15) adversarial law capture.

Measures should include bypass rate, uncovered capability, false admission and refusal, obligation loss, replay divergence, reconstruction success, decision latency, contributor burden, and exception closure.

Open-Source Sovereignty Projection of Jnana

Across the completed P1 corpus, this paper is the **open-source and operating-system sovereignty projection of the Jnana constitutional transformation substrate**. It explains what changes when AI makes proposal generation cheap: transformation authority, capabilities, effects, refusal, reconstruction, provider exit, and contributor governance must become inspectable and contestable. It is not a separate Jnana kernel, a Linux replacement, or evidence that the proposed stack has been implemented.

- **Adjacent work:** *Jnana*
  Supplies: Original governed-genome and constitutional-software architecture.
  Projection boundary: This paper projects its commitments into open-source collaboration.
- **Adjacent work:** *Jnana II: Constitutional Transformation Substrate*
  Supplies: Refined transformation-substrate interpretation.
  Projection boundary: It supplies the architectural center; this paper supplies the public collaboration thesis.
- **Adjacent work:** *Sovereign Software*
  Supplies: Inspectability, reconstructability, substitutability, renegotiation, fork, and exit objectives.
  Projection boundary: Jnana is one proposed realization family for those sovereignty goals.
- **Adjacent work:** *Governance Coverage Gaps*
  Supplies: Detection of unmediated capability and effect paths.
  Projection boundary: Coverage tests whether AI-as-proposer is enforced rather than merely named.
- **Adjacent work:** *From CI/CD to ACG: From Tests to Courts in Agentic Software Development*
  Supplies: Governed development, adjudication, promotion, evidence, and revision lifecycle.
  Projection boundary: ACG supplies a development workflow that can operate through the Jnana projection.

An open-source transformation contribution should bind:

$$\begin{aligned} T = (&project, source, dependencies, proposer, proposal,\\ &requested_capabilities, law, authority, decision,\\ &transformation, effects, evidence, refusal,\\ &reconstruction, exit) \end{aligned}.$$

The proposer can be a human, model, tool, or composite. Proposal origin does not confer maintainer standing. Every effect-producing route, including build hooks,

package scripts, shells, network tools, credentials, services, devices, emergency access, and provider-side operations, must be either mediated, explicitly outside the claim, or recorded as a coverage gap.

Sovereignty is graded. Source availability without build reconstruction, policy portability without dependency availability, or receipts without replaceable services provides only partial exit. Contributor accessibility, governance complexity, schema ownership, and forkability must be tested alongside containment and reproducibility.

The universal GSM can type
`PROPOSES_PROJECT_TRANSFORMATION`, `REQUESTS_CAPABILITY`, `ADMITTED_BY_PROJECT_AUTHORITY`, `LOWERED_THROUGH_JNANA_PROFILE`, `PRODUCES_MEDIATED_EFFECT`, `REFUSED_WITH_BRANCH_OPTION`, `RECONSTRUCTABLE_WITHOUT_PROVIDER`, `EXITS_DEPENDENCY`, `ESCAPES_GOVERNANCE_COVERAGE`, and
`CONTESTED_BY_CONTRIBUTOR`. These relations establish the projection without granting implementation standing.

This integration establishes four editorial commitments: classify the paper as the open-source sovereignty projection of Jnana; adopt the contribution envelope; require graded reconstruction, provider-exit, coverage, and contributor-governance evidence; and keep AI proposal, maintainer authority, project admission, and realized effects structurally separate.

Contribution-to-Realization Contract

The fixed-seed comparison with the consciousness-disambiguation protocol, *Meaning as Binding*, *The Ableman Wheel*, and *The Implementation Codex* sharpens the route from an open-source proposal to a realized effect:

$$C = (proposal, semantic_commitments, project_authority, admission, IC, realization_package, IR, effects, evidence, revision, exit)$$

Meaning as Binding explains how project terms, requirements, authorities, and dependencies become consequential, but the binding becomes institutionally operative only under an admitted project rule and accountable authority. Proposal coherence, popularity, AI fluency, or semantic integration cannot substitute for maintainer or community admission.

The Implementation Codex can package the selected Instance Constitution / Instance Configuration into one or more candidate Instance Realizations / Implementation Realizations with phases, gates, tests, journals, and remainder. It is a realization interface for a Jnana-governed contribution, not proof that the Jnana substrate or proposed change is correct, secure, sovereign, or complete.

The randomized adjacencies to consciousness discourse and the Ableman Wheel both produce
NO_MATERIAL_RELATION. AI proposal handling supplies no evidence about machine consciousness, and a governed software contribution path supplies no physical evidence or construction authority for the Wheel. Either domain could use generic governance tooling without creating a conceptual dependency.

The universal GSM can type
PROPOSES_OPEN_SOURCE_CONTRIBUTION,
BINDS_PROJECT_SEMANTICS, ADMITS_BY_PROJECT_AUTHORITY, SELECTS_INSTANCE_CONFIGURATION,
PACKAGES_IMPLEMENTATION_REALIZATION, EVIDENCES_REALIZED_EFFECT,
PRESERVES_PROVIDER_EXIT,
and
HAS_NO_MATERIAL_RELATION_TO_CONSCIOUSNESS_OR_
WHEEL_CLAIMS.

This randomized review establishes four editorial commitments: adopt the contribution-to-realization contract; make project authority the bridge from semantic proposal to operative commitment; use the Implementation Codex as a non-certifying realization interface; and record no direct consciousness or Wheel relation.

The Contribution-to-Sovereignty Contract

Three retained downstream uses locate the Jnana open-source projection precisely. Sovereign Software supplies reconstructability, dependency control, delegation, provider removal, and effective exit. Meaning as Binding supplies operational and institutional binding while withholding project authority from semantic force. The Implementation Codex packages a selected configuration into a realization path. The consciousness-disambiguation and Ableman Wheel comparisons are explicit nonrelations: constitutional software architecture supplies neither consciousness evidence nor physical apparatus results.

The contribution contract is:

$$\begin{aligned} C = (&proposal, authorship, license, project_authority,\\ &sources, dependencies, capabilities, tests, review,\\ &admission, selected_IC, realization_package, IR,.\\ &effect_boundary, evidence, acceptance, fork, exit,\\ &remainder) \end{aligned}$$

AI may propose, translate, test, explain, or package a contribution. It does not become author, maintainer, rights holder, reviewer, admission authority, or effect authority by doing so. Human approval of one field does not silently adopt every generated dependency, design choice, or claim. Project acceptance, technical

correctness, security, sovereignty, and ecosystem compatibility remain separate postures.

The sovereignty promise should be tested over the contribution's dependency closure: can an authorized community inspect, rebuild, substitute, migrate, fork, and continue without a privileged provider? A portable source archive is insufficient if toolchains, credentials, services, models, data, registries, or update channels remain irreplaceable. Fork rights also require an intelligible governance and migration path, not merely a license grant.

The universal GSM can type
`PROPOSES_OPEN_SOURCE_CONTRIBUTION`,
`ATTRIBUTES_AI_GENERATED_FIELD`, `REVIEWS_UNDER_PROJECT_AUTHORITY`,
`PACKAGES_SELECTED_CONTRIBUTION_IC`, `TESTS_DEPENDENCY_CLOSURE_EXIT`,
`FORKS_WITH_GOVERNANCE_LINEAGE`, `ACCEPTS_WITHOUT_SOVEREIGNTY_PROOF`,
and
`HAS_NO_CONSCIOUSNESS_OR_PHYSICAL_EVIDENCE`.

This reverse-dependency review establishes four editorial commitments: adopt the contribution-to-sovereignty contract; keep AI proposal, authorship, review, admission, and effect roles separate; test inspectability, reconstruction, substitution, migration, fork, and provider exit across dependency closure; and preserve the consciousness and physical-domain nonrelations.

13. Counterevidence and Failure Modes

The proposal should be narrowed or rejected where: (1) open-source history is oversimplified; (2) reproducible builds are treated as universal; (3) existing security and governance mechanisms are ignored; (4) AI-as-proposer is only a label; (5) an agent retains an unmediated shell or filesystem path; (6) proposal output receives author standing; (7) the policy engine becomes an opaque oracle; (8) ordinary law can weaken constitutional invariants; (9) break-glass access is silent; (10) receipts omit partial effects; (11) replay omits dependencies, environment, time, randomness, or hardware; (12) deterministic replay is asserted without closure; (13) canonical MCIR contains a false or biased rule; (14) PEIR or Domain Effects identity is unresolved; (15) a compiler is admitted by itself; (16) supply-chain or bootstrap trust remains ambient; (17) kernel or provider interfaces bypass mediation; (18) lower firmware exceeds the assurance boundary; (19) hashes are confused with behavioral correspondence; (20) provider exit fails because dependencies are unavailable; (21) governance complexity excludes contributors; (22) schema ownership centralizes power; (23) machine-checkable law displaces legitimate social contest; (24) refusal creates unrecoverable denial of service; (25) emergency recovery cannot proceed; (26) adoption levels are conflated; (27) a hypothetical Linux profile is described as implemented; (28) GMB inclusion is treated as standing; (29) constitutional papers are merged by thematic overlap; (30) external technical or historical claims remain unverified; or (31) measured outcomes fail to improve sovereignty or containment.

Open-Source Sovereignty Requires Mediation of Every Relevant Effect Path

Public orientation

Jnana proposes a constitutional transformation substrate for AI-assisted software: AI proposes; governed representations carry commitments; versioned law evaluates them; admitted machinery lowers them; effect boundaries mediate them; and receipts preserve lineage. The proposal extends open-source sovereignty only where its controls actually cover the relevant paths. Source visibility, a policy file, or a receipt does not by itself contain an agent.

Adversarial tests

1. **Ambient-capability attack.** Shells, package managers, networks, credentials, editors, caches, agents, installers, firmware, or humans bypass the declared transformation path.
2. **Representation-bypass attack.** A proposal reaches code or effect without passing through the governed intermediate form and admission gates.
3. **Policy-self-amendment attack.** The same actor or compromised path can alter the law, evaluator, adapter, tests, exceptions, and evidence that constrain it.
4. **Receipt-containment attack.** A complete trace is mistaken for prevention, safe execution, correct effect, or successful rollback.
5. **Replay-closure attack.** Deterministic replay is claimed while tools, dependencies, services, clocks, randomness, hardware, firmware, or side effects remain open.
6. **Provider-exit attack.** Exported artifacts preserve code but lose model behavior, policy semantics, histories, secrets, build state, or practical reconstructability.
7. **Open-source-displacement attack.** Constitutional machinery centralizes contribution, raises participation cost, or weakens existing maintainer and community governance.
8. **Hardware-sovereignty attack.** Inspectable software is described as sovereign beyond unverified chips, firmware, boot chains, peripherals, and manufacturing dependencies.

Claim and evidence ceiling

The paper specifies a constitutional transformation architecture and incremental adoption program. It does not establish complete AI containment, universal mediation, a working operating-system layer, reproducible execution, hardware sovereignty, provider independence, security, usability, lower cost, or a new open-source revolution. Jnana does not replace Git, Linux, maintainers, licensing, testing, review, or social governance.

Every containment claim requires enumerated effect paths, enforced isolation or mediation, bypass testing, exact environment closure, recovery and provider-exit exercises, community and usability evidence, threat modeling, and current systems and open-source research. No implementation, deployment, or external research was performed in this pass.

Reader-facing consequence

The credible promise is that AI-generated transformation can become inspectable, negotiable, refuseable, reconstructable, and portable - but only to the exact boundary where every consequential path has been shown to obey the constitution.

15. Conclusion

The first open-source revolution opened code. An AI-accelerated software culture also needs to open transformation authority: who proposed a change, which law applied, which capability was granted, which effects occurred, what evidence supports the result, and whether users can refuse, reconstruct, fork, and exit.

Jnana offers a coherent proposal for that extension. Its constitutional value does not arise from attaching AI to an operating system. It arises from keeping AI output subordinate to inspectable authority, mediated capability, bounded effect, receipt, and revision.

That promise remains conditional. Jnana becomes a sovereignty mechanism only where its boundaries are actually enforced and its costs, exclusions, bypasses, and dependencies remain visible. The next open-source revolution would not merely make power inspectable. It would make power contestable, reconstructable, and refusable.

Reference and Verification Note

No external source was searched in this pass. Further evidentiary strengthening requires authorized primary-source review of open-source history and governance, reproducible builds, software supply chains, capability security, sandboxing and isolation, Linux build and kernel-module mechanisms, firmware and measured boot, RISC-V and other hardware claims, AI coding agents, and current operating-system AI integration.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Revision.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated

contribution at this stage.

Corpus-Wide Reference Positioning

This paper argues that open source made source code inspectable, modifiable, redistributable, and forkable under declared licenses and social institutions. Its nearest external comparison fields are open sovereign, provenance, security. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Source availability and reproducible builds improve inspectability but do not alone guarantee comprehensibility, practical exit, hardware control, or sustainable maintenance. A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Free Software Foundation. *The Free Software Definition.* https://www.gnu.org/philosophy/free-sw.html **Relationship:** Normative predecessor for user freedoms; the paper may add reconstruction, governance, or hardware-boundary obligations beyond license rights.

[**E2**] Open Source Initiative. *The Open Source Definition.* https://opensource.org/osd **Relationship:** Adjacent licensing baseline; source availability is not treated as sufficient for operational sovereignty.

[**E3**] Martin Kleppmann et al. "Local-First Software: You Own Your Data, in Spite of the Cloud." *Onward! 2019*, pp. 154-178. DOI: 10.1145/3359591.3359737. https://www.inkandswitch.com/local-first/ **Relationship:** Adjacent local-first design program relevant to user custody, offline operation, and collaboration.

[**E4**] Reproducible Builds Project. *Formal Definition of Reproducible Builds.* reproducible-builds.org source **Relationship:** Adjacent reproducibility practice; byte reproduction is compared with, but not equated to, semantic or constitutional continuity.

[**E5**] W3C. *PROV-O: The PROV Ontology.* W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E6**] Santiago Torres-Arias et al. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 2019, pp. 1393-1410. usenix.org source **Relationship:** Predecessor supply-chain lineage system used for comparison; no influence claim is made unless the source head already documents one.

[**E7**] OpenSSF. *Supply-chain Levels for Software Artifacts (SLSA), Version 1.2.* 2026. https://slsa.dev/spec/v1.2/ **Relationship:** Adjacent software-supply-chain assurance standard, especially relevant to provenance and verification boundaries.

Ableman corpus

[**A1**] Adam Ableman. *Sovereign Software.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-24C6159B98023A9C`;
source SHA-256
`6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Jnana.* Ableman Corpus current edition; work identity

AMCS-LOCAL-WORK-ECEAB04A620F163D;
source SHA-256
76D921D2975BDCD25D6DFB506F5E85F7AFDBF4D2CCC65F8C45394121F9D90060.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *The Substrate Migration Principle*. Ableman Corpus current edition; work identity PAPER.PUB.18726868;
source SHA-256
94111190A4C08A4E739FF7E77BF7B3C37C959E47A33F21CD1DAA29668C9B4BA0.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Cold Boot Continuity as a Deployment Primitive*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-3DB3470715D914D0;
source SHA-256
4CBB05E90E4ECA37EDA78C0E1FC1FDD180B2D6E0936831B57379AD7124776D4E.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Reasoning by Way of Receipts*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-04E427F6F4D94454;
source SHA-256
AF86A66CCD6A103DD75208500AE99148911A6453B6DFDF36A954279F21AB8489.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *The Epistemic Lineage Engine*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-E0EC3DC7EABF5060;
source SHA-256
CDD74D9AFAB6532645E68F7F6C9F0F7A9E472D9B8E717290522566FF8DC01F11.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *The Three Hash Rule*. Ableman Corpus current edition; work identity PAPER.PUB.20788617;
source SHA-256
F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety,

novelty, or performance claims.

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Adam Ableman *Semantic Coordinate Identity Tokenization (SCIT).* Zenodo, 2026. doi:10.5281/zenodo.20532070

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23847981